EX15 - Trayectoria NumPy - Arrays, shape y columnas

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex15_numpy_trajectory_analysis.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Representar la trayectoria como un array NumPy de shape (N,3) y operar sobre columnas completas.

Variables / elementos del problema

Nombre	Significado
trajectory_array	matriz [h,v,t]
energy	g*h + 0.5*v**2

Ecuacion o logica en papel

```
row = [h, v, t]
energy = g*A[:,0] + 0.5*A[:,1]**2
```

Comprobacion manual

El primer estado [[0,50,0]] tiene shape (1,3). Un vector [0,50,0] sin doble corchete tendria shape (3,).

2. Lo que ya sabemos y lo nuevo

Ya visto:

- listas
- tuplas
- diccionarios
- simulacion

Nuevo en este ejercicio:

- np.array
- np.empty((0,3))
- shape
- (3,) vs (1,3)
- np.append con axis=0
- slicing [:,0]
- np.max
- vectorizacion

3. Conceptos Python nuevos

1. Array NumPy

Sintaxis: `np.array([[h, v, t]])`

Un array NumPy almacena datos numericos con una shape. Los dobles corchetes crean una fila 2D.

2. shape

Sintaxis: `A.shape`

shape describe la longitud de cada eje. (N,3) significa N filas y 3 columnas.

3. axis=0

Sintaxis: `np.append(A, row, axis=0)`

En este contexto indica que anadimos filas manteniendo las tres columnas.

4. Slicing

Sintaxis: `A[:, 0]`

: selecciona todas las filas y 0 la primera columna.

5. Vectorizacion

Sintaxis: `g*A[:,0] + 0.5*A[:,1]**2`

NumPy aplica las operaciones a todos los elementos de la columna sin un for Python explicito.

4. Codigo paso a paso

Paso 1. Crear matriz vacia

```
trajectory_array = np.empty((0, 3))
```

Tiene cero filas y tres columnas, preparadas para [h,v,t].

Paso 2. Crear primera fila

```
state_0 = np.array([[config["h0"], config["v0"], config["t0"]]])
```

El doble corchete fuerza shape (1,3).

Paso 3. Anadir fila

```
trajectory_array = np.append(trajectory_array, state_0, axis=0)
```

np.append devuelve un array nuevo; por eso reasignamos el resultado.

Paso 4. Seleccionar columnas

```
trajectory_array[:, 0]
trajectory_array[:, 1]
trajectory_array[:, 2]
```

Las columnas representan altura, velocidad y tiempo.

Paso 5. Calcular energia

```
energy = config["g"] * trajectory_array[:, 0] + \
         0.5 * trajectory_array[:, 1]**2
```

La expresion se evalua elemento a elemento y produce un valor de energia por estado.

5. Flujo de ejecucion

```
generate states
 -> append each [h,v,t] row
 -> matrix (N,3)
 -> slice columns
 -> vectorized analysis
```

6. Resultado / comprobacion esperada

El maximo de la columna 0 y el ultimo valor de columnas 1/2 deben coincidir con la solucion obtenida mediante listas.

7. Errores tipicos al empezar

- Crear np.array([h,v,t]) y esperar (1,3).
- Olvidar axis=0.
- Pensar que np.append modifica in-place.
- Confundir fila 0 con columna 0.

8. Si solo recuerdas cinco cosas

- NumPy trabaja con arrays y shapes.
- (3,) no es (1,3).
- A[:,0] significa todas las filas de la columna 0.
- np.append devuelve un nuevo array.
- Vectorizar opera sobre muchos valores a la vez.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.